

Continuous Integration Walkthrough

This walkthrough has two parts: a live run-through of what a broken commit looks like from the moment it's pushed to the moment GitHub stops it from reaching `main`, and a follow-up exercise you can repeat on your own afterwards, on your own copy of the repository.

This walkthrough has two parts: a live run-through of what a broken commit looks like from the moment it's pushed to the moment GitHub stops it from reaching `main`, and a follow-up exercise you can repeat on your own afterwards, on your own copy of the repository.

Looking for a condensed, copy-paste-ready version of section 5 instead of the fully explained one below? See [DEMO_SCRIPT.md](#).

1. What this walkthrough shows

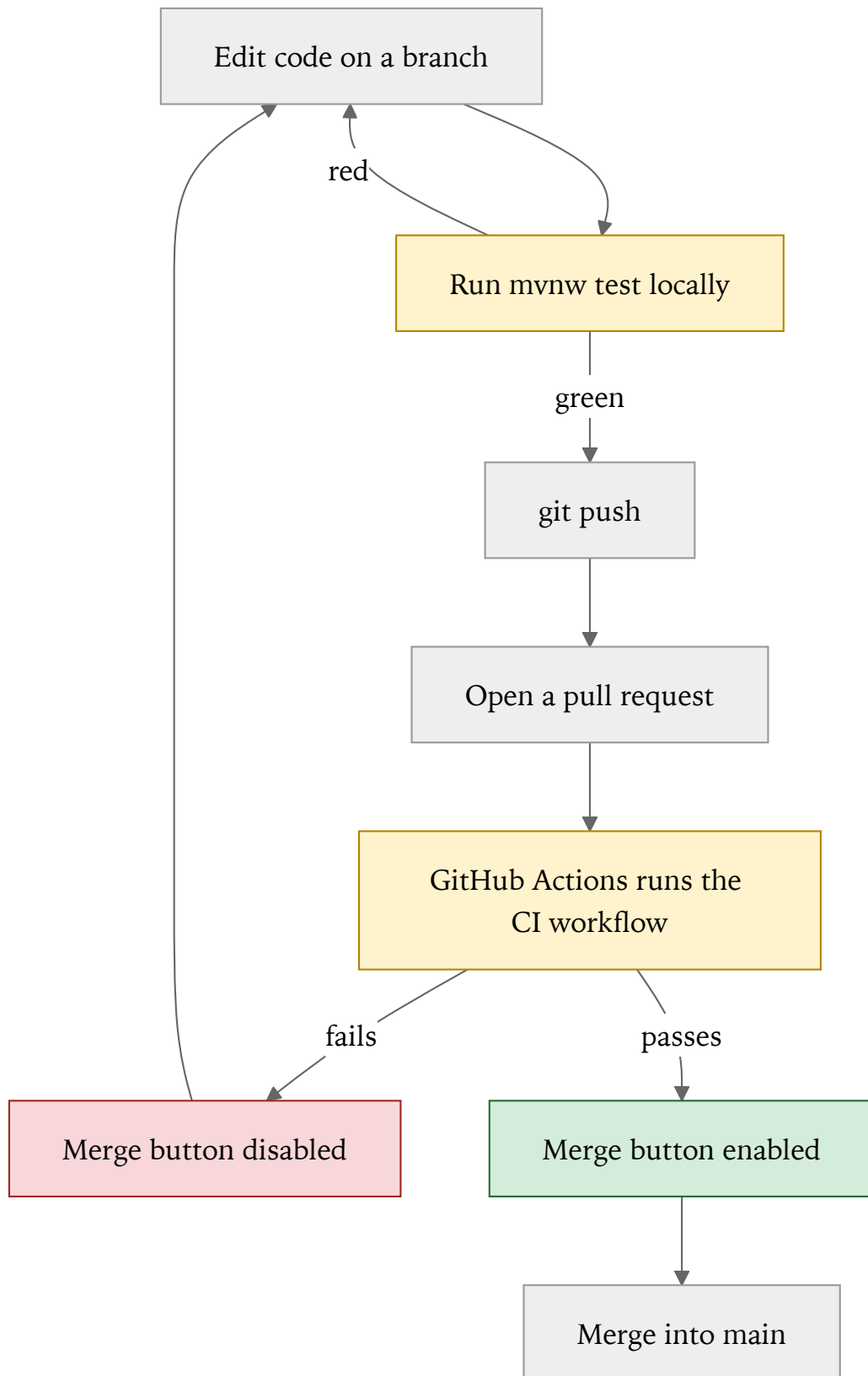
Underneath the stock-portfolio theme, this repository demonstrates four ideas that show up in every professional software team:

1. **Automated tests** catch mistakes without a human having to notice them.
2. **A CI pipeline** (GitHub Actions, in this case) runs those tests automatically every time code is pushed — nobody has to remember to run them by hand.
3. **A pull request (PR)** is the normal way to propose a change to shared code, and it's the natural place to show the result of that automated check.
4. **A branch protection rule** turns "please don't merge broken code" from a polite request into something the platform enforces — the *Merge* button is physically disabled while the check is red.

None of this depends on the specific business logic in this repo. The "stock portfolio" domain is just something concrete and small enough to reason about; it could be a shopping cart, a payroll calculation, anything.

A natural follow-up question: what stops someone from weakening the *tests* instead of fixing the *code*? Section 10 addresses it directly.

The loop we'll walk through, over and over, looks like this:



2. The domain model, in 60 seconds

Two main classes, no framework, no database:

- **Money** (`Money.java`) — an immutable amount of money, always rounded to 2 decimals, implemented as a Java `record`. `Money.of(10).add(Money.of("2.50"))` gives you a new `Money` worth `12.50`.
- **Portfolio** (`Portfolio.java`) — holds a cash balance and stock holdings. `buy(ticker, quantity, price)` spends cash and records shares; `sell(ticker, quantity, price)` releases shares, adds cash back, and returns the realized gain, consuming the oldest purchased shares first (FIFO — first in, first out).

Fifteen JUnit 5 tests, across `MoneyTest` and `PortfolioTest`, pin down this behavior: what a normal purchase does, what happens if you try to buy more than you can afford, what happens if you try to sell more shares than you own, and so on. You don't need to read every test to follow along — just know that they run in a fraction of a second, because there is no database, no network call, and no web server involved. That speed is what makes the rest of this walkthrough possible.

3. Prerequisites

Tool	Notes
JDK 21+	Check with <code>java -version</code> . No separate Maven installation is needed — the checked-in Maven Wrapper (<code>./mvnw</code>) downloads the right Maven version automatically on first use.
Git	Check with <code>git --version</code> .
VS Code	The built-in terminal is all that's needed; the "Extension Pack for Java" is nice to have but not required.
A GitHub account	Needed before section 5: you'll fork the repo (see 6.1) and open pull requests from your own copy — you don't have push access to this repository directly.
GitHub CLI (<code>gh</code>)	Optional. Speeds up creating a pull request from the terminal (<code>gh pr create</code>) instead of the website.

4. Repository tour

COPY THIS BLOCK

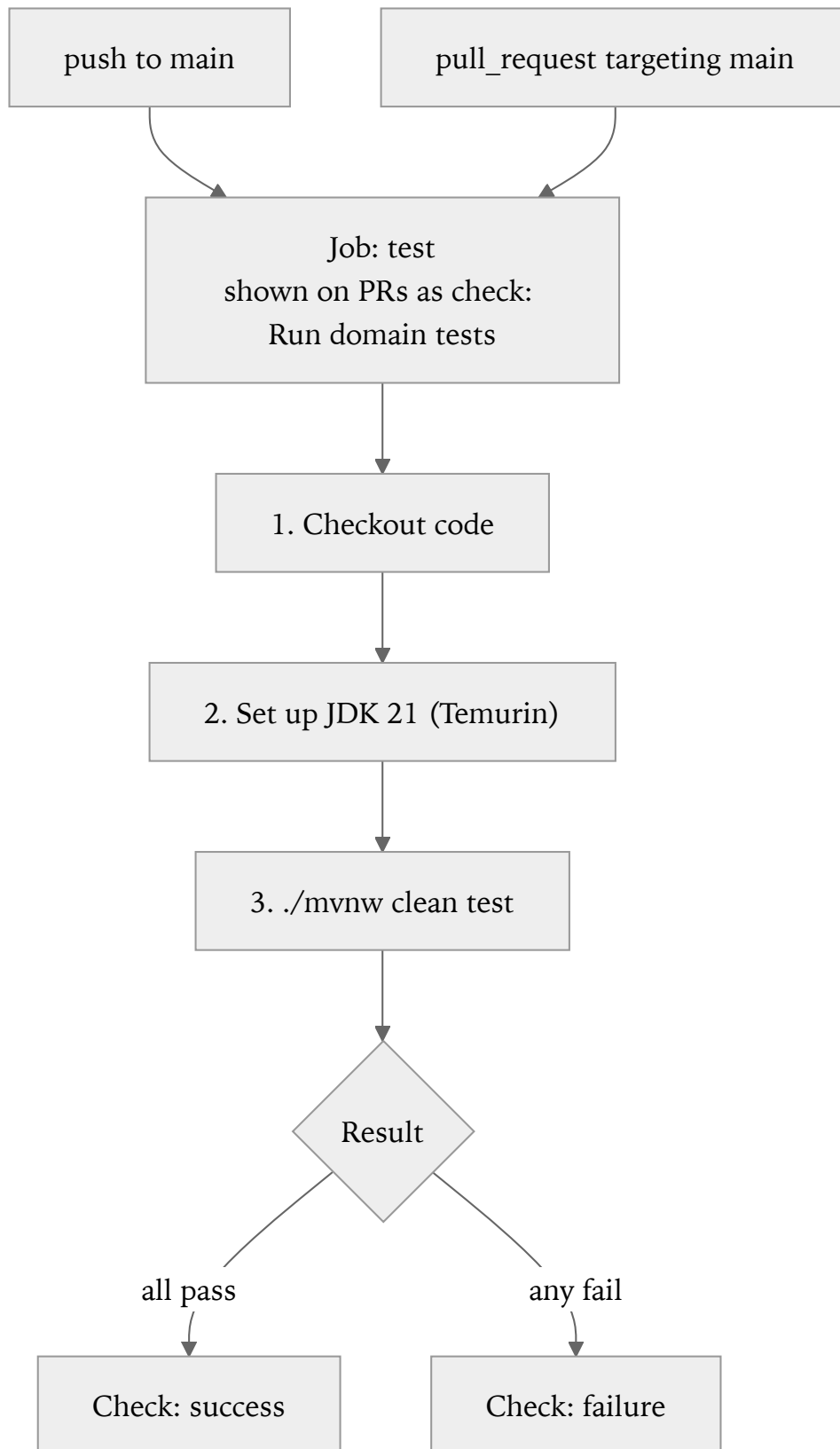
```
ci-demo-java/  
  src/main/java/com/example/portfoliodemo/  
    Money.java           Money value object  
    Portfolio.java       Portfolio: deposit / buy / sell, FIFO lots  
    DomainException.java Base class for domain errors  
    InsufficientFundsException.java  
    InsufficientSharesException.java  
  src/test/java/com/example/portfoliodemo/  
    MoneyTest.java  
    PortfolioTest.java  
  .github/workflows/ci.yml The pipeline definition itself  
  pom.xml                 Project + dependency definition (JUnit 5)  
  mvnw / mvnw.cmd         Maven Wrapper – no local Maven install needed
```

The pipeline is defined in `.github/workflows/ci.yml`, and it's short enough to read end to end:

COPY THIS BLOCK

```
name: CI  
  
on:  
  push:  
    branches: [main]  
  pull_request:  
    branches: [main]  
  
jobs:  
  test:  
    name: Run domain tests  
    runs-on: ubuntu-latest  
    steps:  
      - name: Checkout code  
        uses: actions/checkout@v4  
  
      - name: Set up JDK 21  
        uses: actions/setup-java@v4  
        with:  
          distribution: temurin  
          java-version: "21"  
          cache: maven  
  
      - name: Run tests  
        run: ./mvnw -B -ntp clean test
```

In plain English: every time someone pushes to `main`, or opens/updates a pull request targeting `main`, GitHub spins up a fresh Ubuntu machine, checks out the code, installs a Temurin JDK 21, and runs `./mvnw clean test`. That's the entire pipeline — no deployment step, no infrastructure. This is CI (Continuous **I**ntegration), not CD (Continuous **D**eployment).



The job is named `Run domain tests`. That exact name is what a **branch protection rule** on `main` requires to pass before a pull request can be merged. The rule lives at:

COPY THIS BLOCK

<https://github.com/<owner>/ci-demo-java/settings/branches>

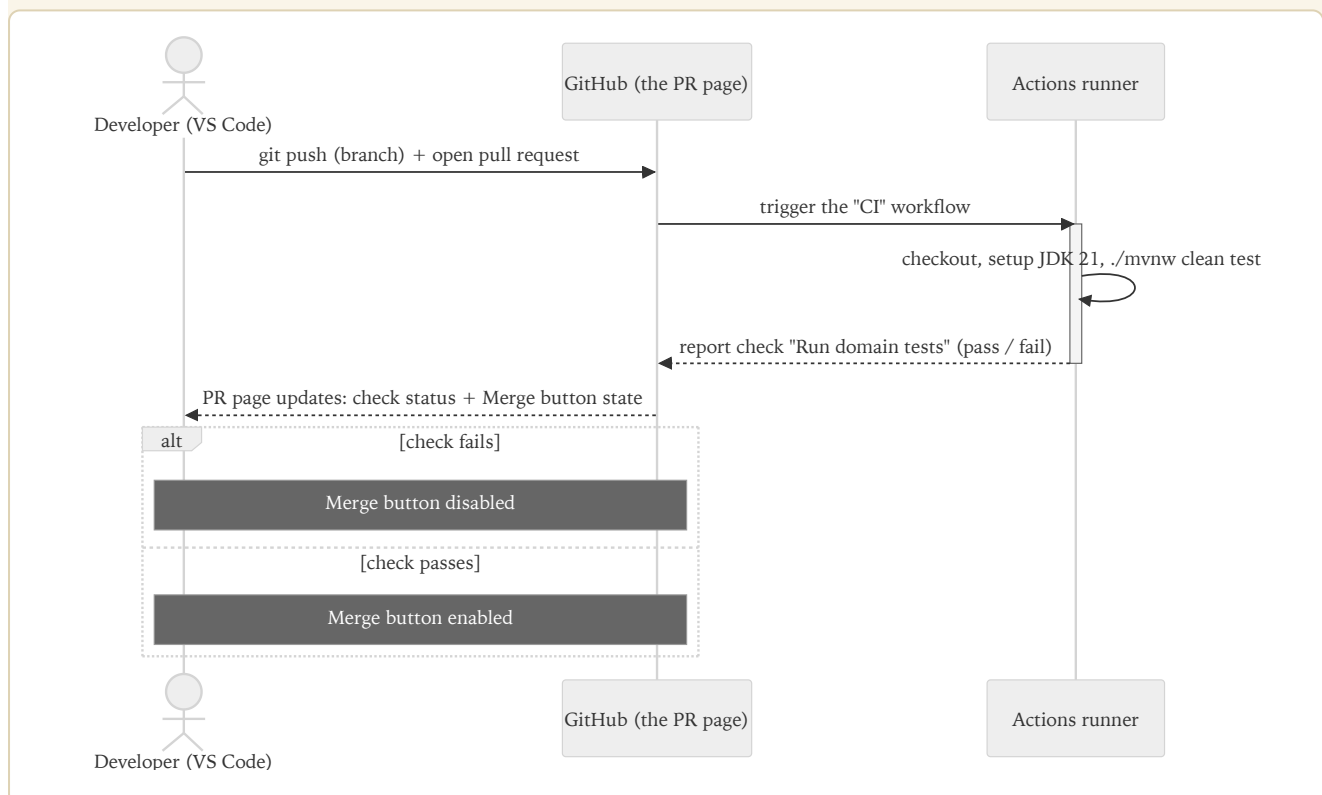
It requires:

- The `Run domain tests` check to pass.
- The branch to be up to date with `main` before merging (this matters — see the troubleshooting section if a check looks green but the PR still won't merge).
- This applies to **everyone**, including repository administrators — there is no bypass.

5. Live walkthrough: breaking the build on purpose

Before continuing: fork this repository and clone your fork — see [6.1 Fork the repository](#) and [6.2 Clone your fork](#) below — then run every command in this section inside your fork. Cloning `alfredorueda/ci-demo-java` directly won't let you push a branch or open a pull request.

The best way to understand a CI pipeline is to watch it catch something. Here's the mental model before diving in — three actors, one round trip:



The only thing that changes between a failing run and a passing run is whether that last message back to GitHub says "pass" or "fail" — everything else in the diagram happens identically either way.

5.1 Starting point

We start from a clean `main` branch, where all 15 tests pass:

COPY THIS BLOCK

```
git checkout main
git pull
./mvnw clean test
```

Tests run: 15, Failures: 0, in well under a second — no Docker, no database, no network call. That speed is what makes the rest of this walkthrough possible: a slow test suite would make this whole loop painful enough that people stop running it.

5.2 Creating a branch and introducing a bug

COPY THIS BLOCK

```
git checkout -b break-the-build
```

Open `src/main/java/com/example/portfoliodemo/Portfolio.java` and find the `buy` method:

COPY THIS BLOCK

```
public void buy(String ticker, int quantity, Money price) {
    Money cost = price.multiply(quantity);
    if (cost.amount().compareTo(cash.amount()) > 0) {
        throw new InsufficientFundsException(
            "Cannot buy %d %s: cost %s exceeds cash balance %s"
                .formatted(quantity, ticker, cost, cash));
    }
    cash = cash.subtract(cost);           // <-- comment out this line
    lots.computeIfAbsent(ticker, t -> new ArrayDeque<>()).addLast(new Lot(quantity, price)
}
```

Comment out the highlighted line, so buying shares no longer deducts the cost from the cash balance — a realistic, relatable bug: forgetting to update a balance after a transaction.

Running the tests locally, **before pushing anything**, already shows the problem:

COPY THIS BLOCK

```
./mvnw clean test
```

COPY THIS BLOCK

```
Tests run: 15, Failures: 2, Errors: 0, Skipped: 0
```

The two failures are `PortfolioTest.buyDeductsCashBalance` and `PortfolioTest.sellIncreasesCashBalance` — one small bug broke two different behaviors, which is exactly why a suite covers more than one scenario instead of a single happy path.

5.3 Pushing the branch and opening a pull request

COPY THIS BLOCK

```
git add -A
git commit -m "Introduce a bug on purpose for the CI demo"
git push -u origin break-the-build
```

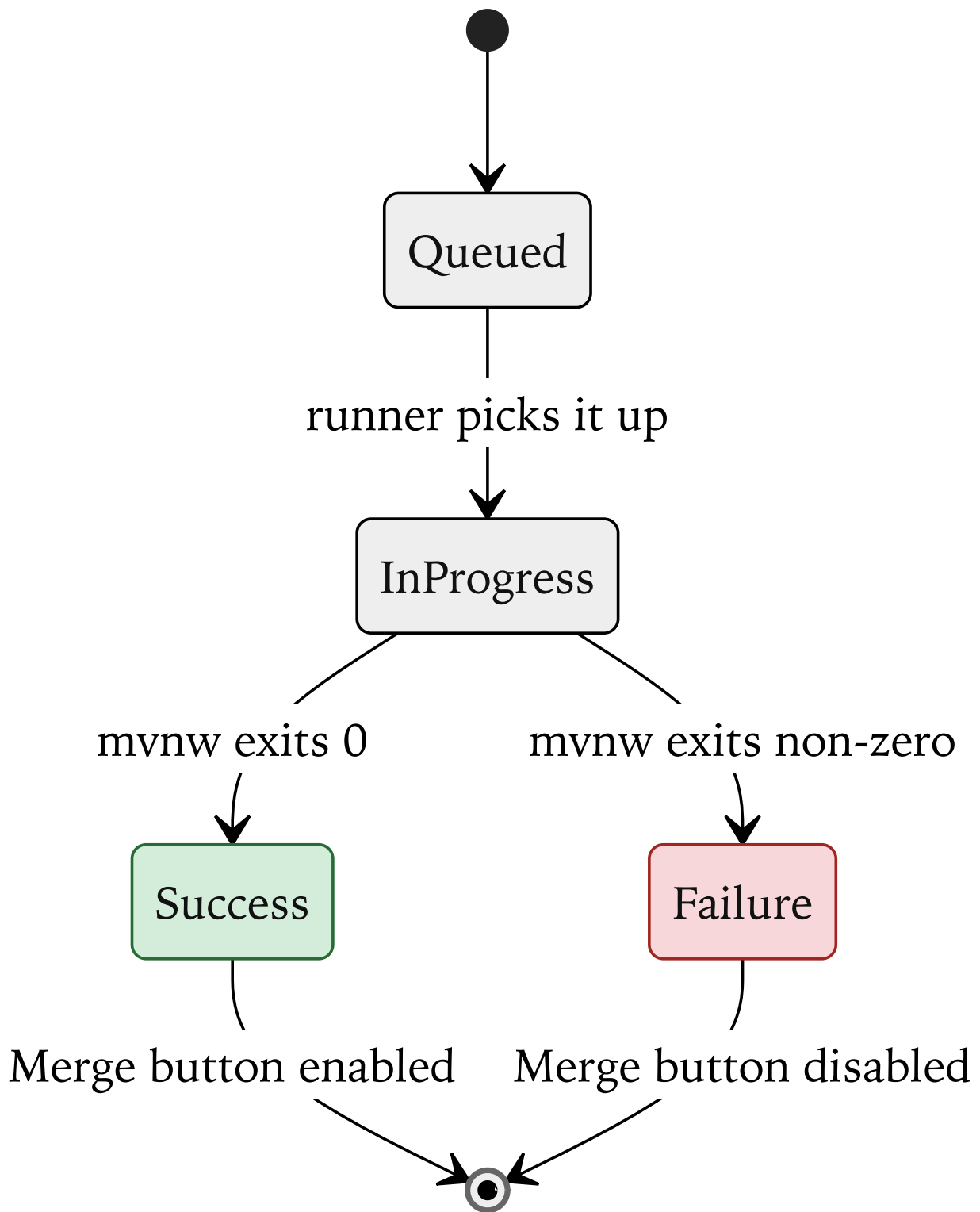
Open a pull request against `main` :

- On the website: click the "Compare & pull request" banner GitHub shows after the push, or go to the repository's **Pull requests** tab → **New pull request**.
- From the terminal, with `gh` installed: `gh pr create --fill`.

This is the moment where, in a real team, this change would be shared for review. Notice it wasn't pushed straight to `main` — the branch protection rule wouldn't have allowed that anyway.

5.4 Watching the check fail

On the pull request page, the check goes through three states, and only one of them lets the PR merge:



Right after opening the PR, the check shows a yellow dot (queued / in progress) next to `Run domain tests` — GitHub is spinning up a fresh virtual machine, downloading dependencies, and starting the JVM, which usually takes 20–40 seconds (a little longer than an interpreted-language pipeline, since there's a build step). A refresh turns it into a red ✕.

Clicking **Details** next to the failed check opens the Actions log, with the same test output that showed up locally a moment ago. CI didn't find something new here — it just proved, automatically and visibly to everyone with access to the repository, what already showed up on one machine.

At the bottom of the PR, the **Merge** button is greyed out, with a message like *"Merging is blocked — Required statuses must pass before merging."* That's not a suggestion: nobody — not even a repository administrator — can merge this pull request until the check turns green.

5.5 Fixing it and watching it go green

Back in the editor, uncomment the line that was disabled:

COPY THIS BLOCK

```
cash = cash.subtract(cost);
```

COPY THIS BLOCK

```
./mvnw clean test    # confirm 15 tests pass, locally, before pushing
git add -A
git commit -m "Fix the cash balance bug"
git push
```

On the pull request page, the check reruns, turns yellow then green, and the **Merge** button becomes active. Merging it (**Squash and merge** is a reasonable default) closes the loop.

5.6 What this shows

- The full loop — write code, get automatic feedback, fix, merge — took a few minutes **only because the test suite is fast and has no infrastructure to spin up**. A slow suite (databases, containers, external services) trains people to stop waiting for it, which is exactly how bugs like this one slip into `main` on real projects.
- Branch protection turns *"please run the tests before merging"* — a request people forget under deadline pressure — into *"you cannot merge until they pass"* — a rule the platform enforces, with no exceptions.
- Everything above is defined as code, in `.github/workflows/ci.yml`. It's reviewed, versioned, and changed through pull requests exactly like the application code it tests.

6. Try it yourself

This section reproduces everything above on your own copy of the repository, so nothing here can affect anyone else. It takes about 15–20 minutes. Already forked and cloned before section 5? Skip ahead to [6.3](#).

6.1 Fork the repository

1. Go to `https://github.com/<owner>/ci-demo-java`.
2. Click **Fork** (top-right corner). This creates a full copy of the repository under your own GitHub account, e.g. `https://github.com/<your-username>/ci-demo-java`.

6.2 Clone your fork

Open a terminal (or the VS Code terminal) and run, replacing `<your-username>`:

COPY THIS BLOCK

```
git clone https://github.com/<your-username>/ci-demo-java.git
cd ci-demo-java
```

6.3 Confirm the baseline is green

macOS / Linux:

COPY THIS BLOCK

```
./mvnw clean test
```

Windows:

COPY THIS BLOCK

```
mvnw.cmd clean test
```

Tests run: 15, Failures: 0. No local Maven installation is required — the wrapper downloads it. Only a JDK (21 or newer) needs to be installed. If the build doesn't run, see [Troubleshooting](#) before continuing.

6.4 Turn on branch protection on the fork

Forking copies the code, but **not** the branch protection rule — that needs to be set up again to see the "merge blocked" behavior on the fork:

1. On the fork, go to **Settings** → **Branches**.
2. Click **Add branch protection rule** (or **Add rule**).
3. Under "Branch name pattern", type `main`.
4. Enable **Require status checks to pass before merging**.
5. In the search box, find and select **Run domain tests**. If it doesn't appear yet, push a commit first — a check only becomes searchable after it has run at least once on the fork — then come back to this step.
6. Enable **Do not allow bypassing the above settings** (the equivalent of "include administrators" — it makes the rule apply to the fork's owner too, not just to other contributors).
7. Click **Create** (or **Save changes**).

6.5 Repeating the break → PR → fix loop

Follow the same steps as [section 5.2 through 5.5](#), on the fork:

1. `git checkout -b break-the-build`

2. Comment out the cash-deduction line in `src/main/java/com/example/portfoliodemo/Portfolio.java`.
3. Run `./mvnw clean test` locally — confirm 2 failures.
4. Commit, push, open a pull request against the fork's own `main`.
5. Watch the check fail, look at the log, notice the disabled **Merge** button.
6. Uncomment the line, run `./mvnw clean test` locally to confirm 15 pass, commit, push again.
7. Watch the check turn green and merge the pull request.

6.6 A few things worth trying afterwards

Not required, but they build real intuition:

- **Break a different test on purpose.** Try changing the FIFO order in `sell` (swap `peekFirst / pollFirst` for `peekLast / pollLast`) and see which test catches it.
- **Add a brand new test.** Write a test for a scenario that isn't covered yet — e.g. selling shares from three different lots in one call — and watch it run automatically on the next push, with zero extra configuration.
- **Push directly to the fork's `main`.** With branch protection on, even a direct push (not through a PR) gets rejected — the same rule at work from a different angle.
- **Temporarily disable the branch protection rule** (Settings → Branches → delete or edit the rule), then repeat the broken-test scenario. The check still turns red — but now the **Merge** button stays active. This separates two ideas that are easy to conflate: *running tests automatically* and *enforcing that they must pass* are two different features, and this just turned one of them off.

7. Glossary

Term	Meaning here
Repository (repo)	The project's folder, tracked by Git, hosted on GitHub.
Commit	A saved snapshot of changes, with a message describing them.
Branch	An independent line of work, starting from some point in the project's history — lets changes happen without touching <code>main</code> until they're ready.
Pull request (PR)	A request to merge one branch into another (usually into <code>main</code>), with a page on GitHub where the diff, comments, and checks are shown.
CI (Continuous Integration)	Automatically building and testing every change, on every push, instead of relying on someone remembering to do it.
Workflow	A pipeline definition file, here <code>.github/workflows/ci.yml</code> .
Job / step	A workflow is made of jobs; each job is made of steps run in order. This repo has one job (<code>test</code>) with three steps.
Status check	The pass/fail result of a job, shown directly on a pull request.
Branch protection rule	A GitHub setting on a specific branch (here, <code>main</code>) that can require checks to pass, forbid force-pushes, etc., before a change is allowed in.
Merge blocked	The state where a pull request cannot be merged because a required condition (here, the <code>Run domain tests</code> check) hasn't been satisfied yet.
Maven Wrapper (<code>mvnw</code>)	A checked-in script that downloads the exact Maven version a project needs, so nobody has to install Maven separately — the Java equivalent of committing a Python virtual environment's requirements file.

8. Troubleshooting / FAQ

The check has been "Queued" for a while. GitHub-hosted runners can take anywhere from a few seconds to about a minute to start, especially at busy times. Refresh the page; there's nothing to fix.

The check is green, but the Merge button is still disabled. This branch protection rule uses a *strict* status check, meaning the branch must also be up to date with the latest `main` before merging (not just passing on its own). GitHub shows an **Update branch** button on the PR — click it, wait for the check to re-run on the updated code, and the Merge button becomes available.

A local test result looks stale right after an edit. Maven's incremental compiler occasionally reuses a class file compiled just before a very quick edit, especially when a fix and a re-run happen within the same second. Run `./mvnw clean test` — the `clean` step removes compiled output first, guaranteeing a fresh build. GitHub Actions is not affected by this, since it always builds from a clean checkout.

`./mvnw` fails with a permissions error (**Permission denied**). On macOS/Linux, the script needs the executable bit set: `chmod +x mvnw`, then try again.

The workflow never runs at all after a push. Check that the push went to a branch with a PR opened against `main` (or straight to `main`) — the workflow only triggers on those events (see the `on:` block in `.github/workflows/ci.yml`). Also double-check the file is still at exactly `.github/workflows/ci.yml` — GitHub Actions only looks in that folder.

A push straight to `main` gets rejected. That's the branch protection rule working as intended — direct pushes to a protected branch are exactly what it's designed to stop. Create a branch and open a pull request instead (see section 6.5).

This is worth sitting with for a moment, because it's easy to read the whole exercise in section 5 as being about a *deliberate* bad change — comment out a line on purpose, watch it get caught — and conclude the protection exists for hypothetical bad actors. In practice, the far more common trigger is far less dramatic: someone starts editing before remembering to create a branch, especially mid-flow or under time pressure, and only notices when `git push` refuses to cooperate. The rule doesn't distinguish between the two cases, and that's the feature, not a limitation — it doesn't ask *why* a change is heading to `main` without review, it just requires the same PR either way.

Nothing is lost when this happens; the rejected `push` means the commit only ever reached the local repository, never `origin`:

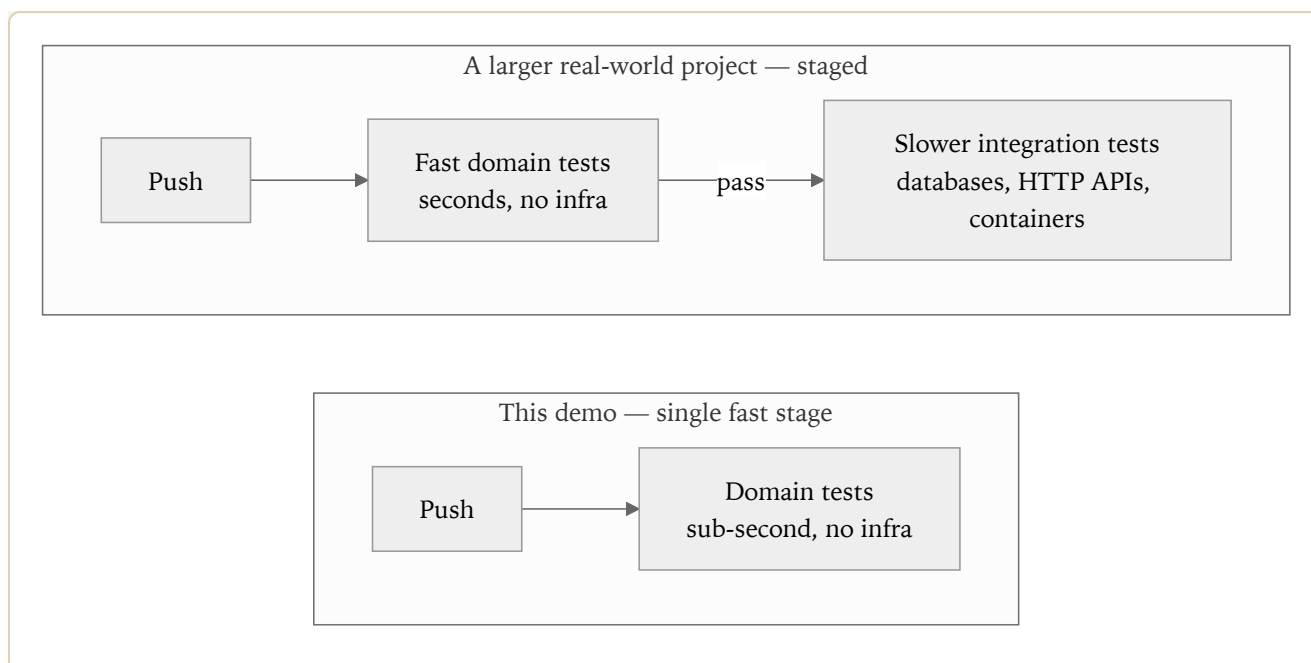
COPY THIS BLOCK

```
git branch break-the-build      # snapshot the current commit onto a new branch
git reset --hard origin/main    # bring local main back in sync with the remote
git checkout break-the-build    # keep working from here
```

`git branch <name>` creates a new branch pointing at whatever commit `main` is on right now, without moving `main` itself — so the work isn't discarded, it's just relabeled. `git reset --hard origin/main` then moves the local `main` branch back to match the remote exactly, discarding the commit *from* `main` (not from `break-the-build`, which now also points at it). From there, section 5.3 onward applies unchanged: push the branch, open a pull request, let the check run.

9. Where to go from here (optional, beyond this walkthrough)

This repository intentionally keeps everything at one speed — pure domain logic, no infrastructure. In a real project, tests usually end up in two tiers: fast, infrastructure-free tests like these, and slower integration tests that talk to a real database or API. A common next step is a **staged pipeline**: a fast job that runs on every single push for instant feedback, followed by a slower job — using `needs:` in the workflow file — that only runs once the fast one has passed, or only on merges into `main`.



The same idea, applied to a much larger, production-shaped codebase, is worth exploring once this smaller version feels familiar.

10. Why a passing check isn't the whole guarantee: protecting test integrity

A question follows naturally from everything above: branch protection guarantees that `Run domain tests` passes before a pull request can be merged — but what stops a contributor from weakening or deleting the very tests that check runs, inside the same pull request that introduces the bug? If the tests themselves can be edited freely, "the check is green" and "the code is correct" are not actually the same statement.

This is a well-documented limitation of CI-based gatekeeping in general, not a flaw specific to this repository: an automated check can only verify that the code satisfies the rules currently encoded in the test suite. It cannot verify that those rules are still the right ones. Engineering organizations address this gap with a combination of practices, layered on top of — not instead of — the CI mechanism this walkthrough demonstrates.

10.1 Mandatory human review

Automated status checks and human review answer different questions, and production-grade branch protection requires both:

- **Require a pull request before merging**, with **required approving review count ≥ 1** , ensures at least one person reads the diff, not just the CI result.
- **Dismiss stale pull request approvals when new commits are pushed** revokes a prior approval the moment new commits — including ones that touch the test files — are pushed, forcing a fresh look.

- Disallowing self-approval prevents the author of a change from also being its only reviewer.

None of this is visible from the check status alone; it is configured separately, under **Settings** → **Branches**, alongside the *Require status checks to pass before merging* rule already covered in section 3.

10.2 CODEOWNERS: routing test changes to the right reviewer

A `CODEOWNERS` file lets a repository designate specific reviewers for specific paths:

COPY THIS BLOCK

```
# CODEOWNERS
/src/test/          @qa-team
```

Combined with the branch protection option **Require review from Code Owners**, this guarantees that *any* pull request touching a test file must be approved by someone from the designated team — typically QA or a senior engineer — regardless of who approves the rest of the change. This is the most direct, widely used answer to "how do we stop people from quietly weakening the tests?": it does not prevent the edit, but it guarantees a specific, accountable reviewer sees it before the change can reach `main`.

10.3 Coverage and mutation testing as an automated gate

Human review does not scale perfectly, so many organizations back it with automated signals that are harder to game than "the suite passes":

- **Code coverage** (JaCoCo, for a Maven/JVM project like this one) can fail the build if a pull request lowers the covered-line percentage below a threshold. This is a coarse signal — a test can assert nothing meaningful and still count toward coverage — but it reliably catches wholesale deletion of test files.
- **Mutation testing** (PIT, Stryker4s) is a stronger, more direct answer to this exact problem. It automatically introduces small, deliberate bugs ("mutants") into the production code and re-runs the suite against each one; a healthy suite should fail on most mutants. A suite that has been weakened — assertions removed, edge cases deleted — lets a much larger share of mutants survive, which mutation testing reports as a falling *mutation score*, even while code coverage stays unchanged.

10.4 Static rules against common tampering patterns

A lightweight, additional CI step can scan a diff for patterns that usually indicate a test was disabled rather than fixed: `@Disabled` annotations, a shrinking total test count relative to `main` without an explicit justification label, or an assertion-free test body. This does not replace review, but it turns an easy-to-miss diff detail into a hard build failure.

10.5 What this means for the mechanism shown in this walkthrough

The branch protection rule demonstrated throughout this document is a necessary layer, not a complete one. It guarantees that the currently defined tests pass before a change reaches `main` — a real and valuable guarantee, and the starting point for everything above. Making sure those tests stay meaningful

is a separate, complementary problem, solved primarily through mandatory human review (CODEOWNERS plus required approvals on test paths) and, in more mature setups, automated integrity signals such as mutation testing. Neither one substitutes for the other.